Context Window Management for Long-Running Agents: Strategies and Tradeoffs

by **Iván Palomares Carrascosa** on June 30, 2026 in **Artificial Intelligence** 0

Share Post Share

In this article, you will learn five practical strategies for managing context windows in long-running AI agent applications, along with the key tradeoffs each approach introduces.

Topics we will cover include:

- Why context windows become a critical bottleneck in agent-based AI systems designed for sustained, autonomous operation.
- Five distinct context management strategies: sliding windows, recursive summarization, structured state management, ephemeral context via RAG, and dynamic context routing.
- The inherent tradeoffs of each strategy, from memory loss and information compression to retrieval blind spots and maintenance complexity.



Introduction

Long-running agents are those capable of exhibiting sustained autonomous execution over time. In these agent-based applications — fueled by interactions with users or other systems in which information snowballs rapidly — the **context window** is a critical bottleneck. Agents and large language models, or LLMs in their abbreviated form, are two sides of the same coin in modern AI systems, so to speak. Accordingly, shifting from "LLMs as prompt-response engines" to "(agent-endowed) LLMs as long-running background processes" turns context windows into a major AI engineering bottleneck.

For all these reasons, managing context windows in the long run requires specific strategies like sliding windows, tiered memory, and dynamic summarization. This article presents five different operational strategies for this, together with their inevitable tradeoffs.

1. Sliding Windows

Think of an AI agent capable of remembering only its last ten minutes of work. Sliding window approaches simply manage memory limits: they drop the oldest messages, making room for the newest ones, with only core instructions being "locked" at the top of the context.

Here is an example of what a sliding window implementation may look like (the code is not intended to be executable on its own; it is shown for illustrative purposes only):

```
def manage_sliding_window(system_prompt, message_history, max_turns=10):
    """Keep the permanent system instructions, and drop the oldest chat turns
    when history gets too long.
```

```
    """
    if len(message_history) > max_turns:
        # Trim history to keep only the 'X' most recent messages
        message_history = message_history[-max_turns:]

    # Always prepend the system prompt so the agent remembers its identity
    return [system_prompt] + message_history
```

While extremely cheap and fast due to no extra AI processing being required, this strategy has a caveat: “digital amnesia”. In other words, if the agent comes across a problem it already tackled an hour before, it will have completely forgotten how to handle it, which may trap it in never-ending loops.

2. Recursive Summarization

Think of this as an image compression protocol like JPEG, but applied to the realm of context windows. Instead of removing the distant past as sliding windows would do, recursive summarization consists of periodically compressing old messages into a summary. This can help keep the overall agent’s “mission and plot” alive throughout long hours of operation, but of course, like in a blurry JPEG file, there is loss of information pertaining to fine details, which leaves the agent with a long-term yet vague memory of past events.

3. Structured State Management

In this strategy, the running chat transcripts are left behind entirely. To replace them, the agent keeps a manageable JSON object that tracks goals, facts, and errors — serving as a structured sort of “scratchpad”. At every turn or step, the raw conversation is discarded, and the AI agent is passed only the core instructions, an updated JSON object, and the current, new input. This is undoubtedly a very token-efficient strategy. However, it heavily depends on the developer’s implemented criteria for what exactly should be tracked. If unexpected yet crucial variables fall outside the predefined schema boundaries, the agent will inevitably ignore them.

This is a simplified example of what the implementation of this strategy could look like:

```
def run_scratchpad_turn(system_prompt, scratchpad_state, new_input):
    """Wipes conversational history entirely. The agent only navigates
    using their core instructions, current state, and new task.
    """
    # Combining the rigid state with the new input into a single prompt
    prompt = f"{system_prompt}\nMEMORIZED STATE: {scratchpad_state}\nNEW INPUT: {new_input}"

    # The AI processes the prompt, returning its next action plus an updated state
    ai_output = call_llm(prompt, response_format="json")

    return ai_output["chosen_action"], ai_output["updated_scratchpad"]
```

4. Ephemeral Context via RAG

The RAG-based strategy offloads everything in the cumulative context to an external database (a vector database in **RAG systems**, as explained here). This is an alternative to forcing an agent to keep its history in active memory, so that a silent search fetches back *only* the most relevant past events into the current prompt, based on relevance. This could theoretically let the agent run indefinitely without context overload issues. There is a downside, however: a retrieval blind spot, particularly if the agent needs to reconnect two apparently unrelated past events. Relying on the retriever and its underlying search policy for this may result in missing relevant context that would otherwise connect important “mental pieces”.

5. Dynamic Context Routing

This strategy is designed to balance capability and cost. It makes two distinct AI models work together. The main agent runs high-frequency, repetitive tasks relying on a faster, cheaper model that manages smaller context windows. Meanwhile, when exceptional events occur — such as failing a task three times in a row — the full raw history is forwarded to a large-context, powerful model, which analyzes the big picture and delivers a cleaner instruction set back to the cheaper model. This is a pretty cost-effective strategy, but the code needed to reliably identify exactly when the cheaper model gets stuck can be extremely difficult to maintain and fine-tune.

Wrapping Up

This article outlined five strategies — and their inevitable tradeoffs — to optimize the management of context windows when working with long-running agent-based AI applications. Bear in mind, though: ultimately, building successful autonomous agent applications isn’t about pursuing the illusion of infinite memory, but rather about building smarter architectures and an underlying logic that helps determine what must be remembered, and what the agent can afford to forget.

Share Post Share

More On This Topic